- AICBD 面試 60 分鐘執行版
  - 一、時間配置與敘事節奏
  - 二、痛點優先序重估：AICBD 真正的戰場
    - 2.1 結論
    - 2.2 一句話串起四痛點
  - 三、3 分鐘擴展版自我介紹
  - 四、Hero Case 1：企業 Copilot 知識工作流
    - 4.1 Situation
    - 4.2 Task
    - 4.3 Action（架構決策點）
    - 4.4 Result
    - 4.5 可複用能力抽象
    - 4.6 遷移到 AICBD
  - 五、Hero Case 2：Deep Research 引擎
    - 5.1 Situation
    - 5.2 Task
    - 5.3 Action（架構決策點）
    - 5.4 Result
    - 5.5 可複用能力抽象
    - 5.6 遷移到 AICBD
  - 六、支援案例：地端 MLOps × 多廠區部署
    - 6.1 精簡敘事
    - 6.2 關鍵論點
    - 6.3 可複用能力
  - 七、深度問答題預演（6 題 + 回答骨架）
    - 7.1 技術面
    - 7.2 跨域面
    - 7.3 行為面
  - 八、白板題預演：若被要求畫系統架構圖
    - 8.1 通用六層骨架（可直接畫）
    - 8.2 口頭陳述
  - 九、反問區（8–10 分鐘，4 題）
    - 9.1 策略層
    - 9.2 實戰層
    - 9.3 個人層
  - 十、禁忌與口訣
    - 10.1 禁忌

- 10.2 口訣
- 十一、最小必打清單（若面試失控，守住這 5 點）

AICBD 面試 60 分鐘執行版

本文件為 60 分鐘面試情境專用，聚焦於「最重要的痛點識別 + 案例佐證」。與 `interview.md`（完整分析文件）互補。

一、時間配置與敘事節奏

段落	時長	面試官預期動作	候選人任務
開場 + 自我介紹	3–5 分	初步判定候選人定位	建立「問題定義型架構師」認知
過往專案深挖	25–30 分	針對履歷條目追問技術細節	交付 2 個 Hero Case（STAR + Transfer）
跨域與挑戰題	10–15 分	測試遷移思維與技術盲點	處理 CV 遷移、Airflow / Kubeflow 落差
候選人反問	8–10 分	評估候選人思考層次	3–4 題高質量反問
結尾	2–3 分	釋出期望 / 時程	簡潔收尾

二、痛點優先序重估：AICBD 真正的戰場

JD 列的四個痛點（參照 `interview.md` 第九章）並非等權重。依 AICBD 三大目標反推：

AICBD 目標	主痛點	次痛點
提升組裝直通率	D. Troubleshooting 自動化	C. 人機標記
提高產品品質	C. 人機標記	B. 小量多樣
縮短新產品導入	B. 小量多樣	A. 資料搜尋

2.1 結論

- **AICBD 真正最重要的痛點：B（小量多樣）+ C（人機標記）** — CV 在製造現場的本質挑戰
- **候選人最強的已驗證實績：A（RAG 搜尋）+ D（RCA 自動化）**
- **敘事核心任務：**用 A + D 的完整落地經驗，證明候選人具備「把模糊需求做成可持續系統」的底層能力；再論述此能力在 B + C 的可遷移性

2.2 一句話串起四痛點

> 這四個痛點共享同一個系統骨架：**問題定義 → 資料結構化 → 模型/檢索層 → 人機協作節點 → 可觀測性 → 持續優化**。我已在 A 和 D 上完整跑通這個骨架，B 和 C 是換節點，不是換骨架。

三、3 分鐘擴展版自我介紹

此為 60 分鐘場次的完整自介，比 30 分鐘版本多出「方法論骨架 + 痛點對應宣告」。

> 我目前在台達擔任企業 AI Agent 平台的架構師與技術負責人。職能上不是做單一模型，而是從問題定義、系統設計、部署、到人機協作整合起來。
>
> 量化成果上，主導過三個代表性專案：
>
> 第一個是企業研發 Copilot，整合 Agentic RAG、需求分析、人機審核，把產品設計文件初稿從 20 天縮短到 2 天，降幅 90%。
>
> 第二個是 Deep Research 引擎，5–20 條平行搜尋代理、自動拆解複雜問題、產出附引用報告，把 RCA 分析時間縮短 30%。
>
> 第三個是底層的企業級 RAG 管線，涵蓋超過 10,000 份文件，採用多階段檢索與權限控制，準確率 80% 以上、延遲 1 秒內，目前在多個廠區落地，活躍使用者超過 20 人。
>
> 方法論上我歸納出一個骨架：問題定義 → 資料結構化 → 模型或檢索層 → 人機協作節點 → 可觀測性 → 持續優化。這個骨架在知識型工作流已經跑通，而它在 AICBD 要解決的 CV 工業應用上是同一套結構——差異只在於資料型態從文字換成影像，其他層級可以遷移。

> 所以我對這個職缺的興趣，不是來轉行做 CV 模型，而是把一套已驗證的企業 AI 落地方法，應用到製造現場。

四、Hero Case 1：企業 Copilot 知識工作流

佐證痛點：A（資料搜尋不易）+ 遷移至 B（新產品導入資料重用）

4.1 Situation

- 場景：企業研發部門的產品設計流程
- 痛點：工程師寫設計文件初稿需 20 天，主要時間消耗在查找歷史規格、RCA、設計準則，以及重複確認需求細節
- 既有系統：文件散落於 SharePoint、PDF、Wiki、內部報告，無統一檢索；資安合規要求文件級權限控制

4.2 Task

將初稿產出時間降至可接受範圍，同時：

- 不犧牲引用正確性（工程文件不容許幻覺）
- 滿足資安合規（權限不可穿透）
- 現場可信任（需可追溯、可驗證）

4.3 Action（架構決策點）

第一步：問題定義 把「寫文件慢」拆成三層：資料可得性、知識組織度、生成可信度。不同層級對應不同解法，避免一把 LLM 解決所有事。

第二步：RAG 管線設計

- 10,000+ 文件 ingestion + metadata normalization
- 多階段檢索：稀疏（BM25）+ 稠密（向量）+ 重排序（reranker）
- 文件級權限控制：檢索層強制過濾，不靠 prompt 約束
- 對話感知 context management：跨輪追蹤引用與約束

第三步：人機協作節點

- 生成時強制引用來源
- TRIZ + 蘇格拉底推理層：結構化提問，讓工程師在生成前釐清模糊需求
- 人工審核工作流：重要節點需簽核

第四步：可觀測性

- Trace、token 用量、工具成功率、幻覺偵測
- 跨廠區效能指標追蹤

4.4 Result

指標	數值
文件初稿時間	20 天 → 2 天（-90%）
檢索準確率	> 80%
查詢延遲	< 1 秒
活躍使用者	> 20 人
任務完成率	≥ 80%

4.5 可複用能力抽象

1. 文件異質性 × 權限複雜度 × 低延遲 的三重約束解法
2. 問題定義先於模型選型（先分層，再分工）
3. 人機協作是系統設計元件，不是事後補丁

4.6 遷移到 AICBD

> 新產品導入 (NPI) 速度慢的根本原因之一，是歷史機種的規格、不良案例、SOP 無法快速檢索與比對。這套 RAG + 權限 + 觀測性的管線可以直接遷移到 AICBD 的機種知識庫，讓新產品導入時可以秒級調出相似機種的 CV 模型、標記規則、缺陷樣態。痛點 A 的解法是痛點 B 的前置條件。

五、Hero Case 2：Deep Research 引擎

佐證痛點：D（Troubleshooting 自動化）+ 遷移至 C（不良標記人機協作）

5.1 Situation

- 場景：工程師做 RCA（Root Cause Analysis）時，需跨多個來源查找、比對、綜合
- 痛點：單次 RCA 涉及 5–10 個資料源，工程師人工搜尋與彙整平均耗時數小時
- 既有流程：全靠經驗，新進工程師難以複製資深人員的排查路徑

5.2 Task

自動化高重複性的搜尋與比對工作，讓工程師聚焦在最終判斷，同時：

- 必須附引用（不可生成無根據結論）
- 必須可追溯（哪個代理搜到、用了什麼工具）
- 必須容錯（部分來源失效時不能整體崩潰）

5.3 Action（架構決策點）

第一步：問題拆解器 輸入複雜工程問題後，系統先做問題分解：拆成 5–20 個子問題，每個子問題配對搜尋策略。

第二步：平行搜尋代理

- 5–20 條代理並行執行
- 每個代理負責一個子問題 + 一組工具
- 跨來源擷取（內部文件、歷史 RCA、MES 紀錄、外部技術資料）

第三步：評分與綜合層

- 對每條搜尋結果做相關性評分
- 重排序後進入 LLM 綜合層
- 產出附來源引用的結構化報告

第四步：System 1 / System 2 雙系統

- 簡單問題走 System 1（快取回應）
- 複雜問題走 System 2（多步驟鏈式推理 + 工具編排）
- 依問題複雜度評分動態切換

第五步：可觀測性

- Agent trace 日誌
- 每條代理的工具成功率追蹤
- 幻覺偵測：引用與生成不一致時觸發告警

5.4 Result

指標	數值
RCA 分析時間	-30%
平行搜尋代理數	5–20
文件引用覆蓋率	每份報告附完整 citation

5.5 可複用能力抽象

1. 複雜任務拆解 → 平行執行 → 綜合收斂 的通用框架
2. 人機協作位置：工程師做最終判斷，代理做重複性搜尋比對
3. 可觀測性從 Day 1 內建，非事後補救

5.6 遷移到 AICBD

> 不良標記本質上也是一種「複雜判斷拆解 + 高重複性比對」任務。AI 預標 + 專家複核的分工，和 Deep Research 的「代理搜尋 + 工程師判斷」是同一個骨架。差異只是工具節點從「文字檢索器」換成「CV 模型」，人機協作層、confidence threshold、feedback loop 的設計是共用的。痛點 D 的骨架直接解痛點 C。

六、支援案例：地端 MLOps × 多廠區部署

佐證痛點：**B**（小量多樣）直接對應

6.1 精簡敘事

- 從 0 設計地端 MLOps 系統，重構雲端 SaaS 功能滿足資安合規
- 在多個製造廠區部署領域驅動的 ML 模型
- 深入現場與使用者對接，將產線痛點轉化為 ML 方案

6.2 關鍵論點

> 多廠區部署的本質就是「小量、多樣、異質」。每個廠區的資料規模都不大，但跨廠區差異極大——料件型號、光源、作業員習慣、產線節拍都不同。這個場景根本無法靠單一大模型解決，必須靠資料策略、任務切分、active learning、domain adaptation。這正是 AICBD 的 CV 模型在工廠導入時會遇到的同一類挑戰。

6.3 可複用能力

- 小樣本下的任務切分策略
- 跨場域模型複用與差異化 fine-tune 的取捨
- 現場使用者對接、痛點轉譯 SPEC 的節奏

七、深度問答題預演（6 題 + 回答骨架）

7.1 技術面

Q1: RAG 準確率只有 80%，那 20% 怎麼處理？ 骨架：引用機制 → 人工審核節點 → feedback loop → 持續微調 reranker。強調「80% 不是終點，是持續優化的起點」。

Q2: Deep Research 的 5–20 代理並行，怎麼控制成本與品質？ 骨架：複雜度評分決定代理數、工具成功率作為調度依據、hallucination detection 作為品質守門。

Q3: 你的 MLOps 架構怎麼對應 Airflow / Kubeflow？ 骨架：DAG、依賴、重試、監控四要素已在既有系統實作，工具切換屬執行層。主動提：Airflow 適合 batch pipeline，Kubeflow 適合模型訓練管線，工具選擇看場域需求。

7.2 跨域面

Q4: 你沒做過 CV，怎麼證明能做 AICBD？ 骨架：見第二章結論 + 第四章 4.6 遷移論述。強調「模型是節點，架構是可遷移的」。避免辯護型回應。

Q5: 從 Agent 平台轉 CV 新事業，動機是什麼？ 骨架：(1) 既有架構能力在 CV + 製造場域有更大槓桿 (2) 從知識工作流拓展到實體製造流程 (3) AICBD 的問題結構和我擅長的問題定義 + 系統整合高度對應。

7.3 行為面

Q6: 講一個你推動專案遇到阻力的例子？ 骨架：2024 LLM 浪潮前主動提案 Copilot，組織尚無共識時如何建立早期價值。STAR 結構，重點放在「問題定義能力如何讓沒有預算的專案先做出原型再爭取支持」。

八、白板題預演：若被要求畫系統架構圖

預期面試官可能要求即席繪製「若你負責 AICBD 某個 CV 專案，系統架構怎麼設計」。

8.1 通用六層骨架（可直接畫）

```
[資料層]  MES / ERP / 影像採集 / 標記資料庫
    ↓
[Ingestion]  正規化 / 權限標註 / metadata
    ↓
[模型與檢索層]  CV 模型 + 歷史案例檢索 + 相似度比對
    ↓
[推理與決策層]  置信度評分 / 規則 + LLM 綜合判斷
    ↓
[人機協作層]  預標 / 複核 / feedback loop / SOP 調用
    ↓
[可觀測性層]  trace / 模型漂移監控 / 準確率追蹤 / 告警
```

8.2 口頭陳述

> 這張圖不是憑空畫的，而是我在 Copilot + Deep Research 上已經跑通的六層結構。搬到 CV 場景時，模型層從「文字檢索」換成「視覺模型」，其他五層的設計原則一樣。重點不在畫得多華麗，而在每一層我都能講清楚為什麼是這樣設計。

九、反問區（8–10 分鐘，4 題）

60 分鐘的場次允許 4 題反問，每題都要有資訊取得 + 思考層次展現的雙重目的。

9.1 策略層

Q1: AICBD 目前的 AI 視覺方案，最大瓶頸是在資料面、模型面、還是導入與流程整合面？ 目的：讓面試官開口講真實痛點，驗證候選人的痛點優先序判斷。

Q2: 現階段的專案是偏單點導入，還是已開始往平台化、可複製能力建設前進？ 目的：展現架構師視角，並探測該職位的技術天花板。

9.2 實戰層

Q3: 在標記、驗證與現場導入上，人和 AI 的分工目前是怎麼設計的？有沒有遇到現場不信任的情境？ 目的：展現對 human-in-the-loop 的理解，並觸發面試官分享實戰問題。

9.3 個人層

Q4: 如果這個角色加入後，前 6 個月最希望解決的問題是什麼？ 目的：直接取得期望值，也展現結果導向。

十、禁忌與口訣

10.1 禁忌

避免	原因
堆砌工具名詞（Kubeflow、Airflow、LangChain…）	這是 Junior 表現，Senior 講架構決策
Vision 超過 30 秒	面試官要看過往，不是看未來
形容詞多於數字	「我很擅長」換成「20→2 天」

避免	原因
解釋 LLM / RAG 基礎概念	面試官是 AI 部門主管
對 CV 經驗不足做辯護	直接用「模型是節點，架構是可遷移的」帶過
講超過三層縮進的故事	Case 保持 STAR 結構，不要補充條件

10.2 口訣

> 先定位，再數字，再遷移，再 case。 問題先定義，資料再整理，AI 才部署。

十一、最小必打清單（若面試失控，守住這 5 點）

1. ✅ 自我定位：問題定義型 AI 架構師
2. ✅ 四個量化數字：20→2 天 / RCA -30% / RAG >80% / 多廠區 >20 用戶
3. ✅ 一個 Hero Case 講完整（Copilot 或 Deep Research 擇一）
4. ✅ 遷移論述一句：模型是節點，架構是可遷移的
5. ✅ 至少 2 個反問，其中 1 個問真實痛點